



Aristotle University of Thessaloniki

Markov Chains

Using Communicating Classes and Tarjan's Algorithm

Author:

Georgios Koufetidis (AEM 4432)

Contents

1	Introduction	2
2	Mathematical Background	2
3	Graph Representation	3
4	Tarjan's Algorithm	3
5	Implementation	4
6	Example	4
7	Comparison with an AI Solution	5
7.1	Analytical Framework for Markov Chain State Classification	5
7.2	Characterization of Recurrence	6
7.3	Characterization of Transience	6
8	Conclusion	6
9	Python Code	6

1 Introduction

A Markov Chain is a stochastic process that moves between states according to fixed transition probabilities. If the chain is currently in state i , the probability of moving to state j is denoted by p_{ij} . These probabilities are stored in a transition matrix P .

The objective of this work is to develop a program that receives a finite transition matrix as input and determines whether each state is recurrent or transient. If a state is recurrent, the program must also identify the recurrent class to which it belongs.

The solution uses graph theory. Each Markov state is represented as a node, and a directed edge $i \rightarrow j$ is added whenever $p_{ij} > 0$. Then, the communicating classes of the Markov Chain are found as strongly connected components of this graph.

2 Mathematical Background

Let

$$S = \{0, 1, \dots, n-1\}$$

be the finite state space of a Markov Chain. The transition matrix is

$$P = \begin{pmatrix} p_{00} & p_{01} & \cdots & p_{0,n-1} \\ p_{10} & p_{11} & \cdots & p_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n-1,0} & p_{n-1,1} & \cdots & p_{n-1,n-1} \end{pmatrix}.$$

Each entry satisfies

$$p_{ij} \geq 0,$$

and every row satisfies

$$\sum_{j=0}^{n-1} p_{ij} = 1.$$

A state j is reachable from state i if there exists an integer $k \geq 0$ such that

$$(P^k)_{ij} > 0.$$

Two states i and j communicate if i can reach j and j can reach i . This is written as

$$i \leftrightarrow j.$$

Communication partitions the state space into disjoint communicating classes.

A communicating class C is called closed if no transition from a state in C leads to a state outside C . Therefore,

$$C \text{ is closed} \iff p_{ij} = 0 \quad \text{for all } i \in C, j \notin C.$$

For a finite Markov Chain, the key result is:

A state is recurrent if and only if it belongs to a closed communicating class.

Thus, the problem becomes computationally equivalent to finding communicating classes and checking which of them are closed.

3 Graph Representation

The transition matrix is converted into a directed graph $G = (V, E)$. The vertices are the states of the Markov Chain:

$$V = S.$$

A directed edge is added according to the rule

$$(i, j) \in E \iff p_{ij} > 0.$$

Therefore, every positive transition probability becomes a possible movement in the graph.

For example, if

$$P = \begin{pmatrix} 0.5 & 0.5 & 0 \\ 0.2 & 0.8 & 0 \\ 0 & 0.3 & 0.7 \end{pmatrix},$$

then the corresponding graph is

$$0 \rightarrow 0, \quad 0 \rightarrow 1,$$

$$1 \rightarrow 0, \quad 1 \rightarrow 1,$$

$$2 \rightarrow 1, \quad 2 \rightarrow 2.$$

This graph representation is useful because communicating classes are exactly the strongly connected components of the directed graph.

4 Tarjan's Algorithm

Tarjan's algorithm is used to find the strongly connected components of a directed graph. A strongly connected component is a maximal set of nodes such that every node can reach every other node in the same set.

The algorithm is based on a depth-first search. During the search, each node v is assigned two values:

$$\text{index}(v)$$

and

$$\text{lowlink}(v).$$

The value $\text{index}(v)$ is the discovery order of node v . The value $\text{lowlink}(v)$ is the smallest discovery index that can be reached from v through the current active search path.

Initially,

$$\text{lowlink}(v) = \text{index}(v).$$

As the algorithm explores outgoing edges, the lowlink value is updated. If a node can reach an earlier node that is still active in the search, then the lowlink value decreases. This is how cycles are detected.

A stack is used to store nodes that have been visited but not yet assigned to a component. When the algorithm finds a node satisfying

$$\text{lowlink}(v) = \text{index}(v),$$

then v is the root of a strongly connected component. The algorithm pops nodes from the stack until v is removed. These popped nodes form one communicating class.

The time complexity of Tarjan's algorithm is

$$O(|V| + |E|),$$

where $|V|$ is the number of states and $|E|$ is the number of positive transition probabilities.

5 Implementation

The implementation consists of four main steps:

1. Validate the transition matrix.
2. Convert the matrix into a directed graph.
3. Apply Tarjan's algorithm to find communicating classes.
4. Check whether each communicating class is closed.

6 Example

Consider the transition matrix

$$P = \begin{pmatrix} 0.5 & 0.5 & 0 & 0 \\ 0.2 & 0.8 & 0 & 0 \\ 0 & 0.3 & 0.4 & 0.3 \\ 0 & 0 & 0.6 & 0.4 \end{pmatrix}.$$

The graph contains the transitions

$$0 \rightarrow 0, \quad 0 \rightarrow 1,$$

$$1 \rightarrow 0, \quad 1 \rightarrow 1,$$

$$2 \rightarrow 1, \quad 2 \rightarrow 2, \quad 2 \rightarrow 3,$$

$$3 \rightarrow 2, \quad 3 \rightarrow 3.$$

Tarjan's algorithm finds two communicating classes:

$$\{0, 1\}$$

and

$$\{2, 3\}.$$

The class $\{0, 1\}$ is closed because there is no transition from either state 0 or state 1 to a state outside the class. Therefore, states 0 and 1 are recurrent and belong to the same recurrent class.

The class $\{2, 3\}$ is not closed because

$$p_{21} = 0.3 > 0.$$

Thus, the chain can leave the class $\{2, 3\}$ and move to state 1. Therefore, states 2 and 3 are transient. The classification is shown in Table 1.

State	Classification	Recurrent Class ID
0	Recurrent	1
1	Recurrent	1
2	Transient	—
3	Transient	—

Table 1: Classification of the states for the example Markov Chain.

7 Comparison with an AI Solution

For the following solution, the AI model that we used was Gemini.

7.1 Analytical Framework for Markov Chain State Classification

To rigorously classify the states of a Markov chain defined by a transition matrix $P \in \mathbb{R}^{n \times n}$, we utilize an analytical framework based on the theory of directed graphs and spectral reachability. The procedure is formalized through three distinct phases: We define the reachability matrix R using the k -step transition probabilities. State i reaches state j (denoted $i \rightarrow j$) if there exists an integer $k \geq 0$ such that $(P^k)_{ij} > 0$. Analytically, we compute the boolean reachability matrix R such that:

$$R_{ij} = \mathbf{1} \sum_k 0^{n-1} (P^k)_{ij} > 0 \quad (1)$$

Two states i and j belong to the same communication class C if and only if $i \leftrightarrow j$, which is satisfied when $R_{ij} = 1$ and $R_{ji} = 1$. This defines an equivalence relation that partitions the state space S into disjoint sets $C_1, C_2, \dots, C_m$.

7.2 Characterization of Recurrence

For each communication class C , we define the escape probability $\mathbb{P}(C \rightarrow C^c)$ as the probability that the process transitions from any state in C to a state in the complement set $C^c = S \setminus C$. The class C is defined as **closed** if:

$$\sum_{i \in C} \sum_{j \in C^c} P_{ij} = 0 \quad (2)$$

In a finite state Markov chain, a closed communication class is necessarily **recurrent**. This implies that the probability of the process returning to any state in C , given that it starts in C , is exactly 1.

7.3 Characterization of Transience

Conversely, if a communication class C is not closed, there exists at least one state $i \in C$ and at least one state $j \in C^c$ such that $P_{ij} > 0$. This implies:

$$\exists i \in C, \exists j \in C^c : P_{ij} > 0 \implies \sum_{i \in C} \sum_{j \in C^c} P_{ij} > 0 \quad (3)$$

In this case, the process possesses a non-zero probability of escaping the class C and never returning, rendering the states within C **transient**. By analyzing the sub-matrix of P restricted to the transient states, one can further derive the fundamental matrix $N = (I - Q)^{-1}$, where Q is the transition sub-matrix, to determine the expected time until absorption into a recurrent class.

8 Conclusion

This report presented a program for classifying states in a finite Markov Chain as recurrent or transient. The transition matrix was first converted into a directed graph. Then, Tarjan's algorithm was applied to find the strongly connected components, which correspond to the communicating classes of the Markov Chain.

After the communicating classes were identified, each class was checked to determine whether it was closed. States in closed communicating classes were classified as recurrent and assigned a recurrent class ID. States in non-closed communicating classes were classified as transient.

Compared with a typical AI-generated solution, this implementation is less complex, without using a complex algorithm and provides clear mathematical connection between Markov Chains.

9 Python Code

All code used to generate the results, figures, and simulations in this report is publicly available at:

[Project GitHub Repository](#)